Top PyTorch Functions CheatSheet

{ik} INTERVIEW KICKSTART

What is PyTorch?

PyTorch is an open-source deep learning framework developed by Facebook's AI Research lab (FAIR). It provides a flexible and dynamic computational graph system, making it popular for research and production in machine learning and deep learning. PyTorch is known for its intuitive and easy-to-use API, which makes it a favorite among researchers and developers for building and training neural networks.

- Official Documentation - https://pytorch.org/docs/stable/index.html
- PyTorch Ecosystem Tools - https://pytorch.org/ecosystem/
- Github Repo - https://github.com/pytorch/pytorch.github.io
- PyTorch Library List - https://github.com/bharathgs/Awesome-pytorch-list

Features of PyTorch

1. **Tensors and Dynamic Computational Graphs -** PyTorch uses tensors, which are similar to NumPy arrays, but can also run on GPUs for faster computation. The dynamic computational graph system allows for flexible model building, where the graph is built on-the-fly during execution.

2. **Autograd -** PyTorch's automatic differentiation library, autograd, automatically computes gradients, which are essential for backpropagation during neural network training. This feature simplifies the process of defining and training complex models.

3. **TorchScript -** TorchScript allows you to transition from research to production by providing a way to create serializable and optimizable models. This is done by scripting your models, making them portable and able to run independently of Python.

4. **Optimizers and Loss Functions -** PyTorch offers a wide range of built-in optimization algorithms and loss functions, which are essential for training neural networks effectively.

5. **Rich Ecosystem -** PyTorch has a rich ecosystem that includes libraries for computer vision (TorchVision), natural language processing (TorchText), and reinforcement learning (TorchRL). This ecosystem provides tools and pre-trained models that simplify the development process.

6. **Support for GPU Acceleration -** PyTorch seamlessly integrates with CUDA, enabling models to leverage GPU acceleration for faster computation, which is crucial for deep learning tasks.

Advantages of PyTorch

1. **Ease of Use**: Intuitive API and dynamic graphs for flexibility.
2. **Community Support**: Widely adopted in academia and industry.
3. **Python Integration**: Works seamlessly with Python libraries.
4. **Production Ready**: Tools for model deployment and scalability.
5. **Interoperability**: Supports exporting models to other frameworks.

Top PyTorch Functions

1. Tensors

a. **Creation -** Tensors are the fundamental building blocks of PyTorch. They are similar to NumPy arrays but can run on GPUs for acceleration.

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.zeros(3)
z = torch.ones(3)
```

b. **Basic Operations** - PyTorch supports a variety of basic tensor operations that are similar to NumPy.

```
a = torch.add(x, y)
b = torch.sub(x, y)
c = torch.mul(x, y)
d = torch.div(x, y)
```

2. **Tensor Reshaping and Visualization**

a. **is_contiguous():** Checks if a tensor is contiguous in memory.

```
t = torch.tensor([[0, 1, 2], [3, 4, 5]])
print(t.is_contiguous())  # Output: True
```

b. **view() vs reshape():** Both reshape tensors, but view() requires the tensor to be contiguous.

```
t = torch.randn(2, 3)
t_view = t.view(3, 2)
t_reshape = t.reshape(3, 2)
```

c. **-1 trick:** Automatically infers the dimension size.

```
t = torch.randn(2, 3, 4)
t_reshaped = t.view(-1, 6)

# Automatically infers the correct size
```

d. **squeeze():** Removes dimensions of size 1.

```
t = torch.zeros(2, 1, 3, 1)
t_squeezed = t.squeeze()
```

e. **unsqueeze():** Adds a dimension of size 1.

```
t = torch.randn(3, 4)
t_unsqueezed = t.unsqueeze(1)
```

f. **transpose():** Permutes the dimensions of a tensor.

```
t = torch.randn(2, 3)
t_transposed = t.transpose(0, 1)
```

g. **`concat()`:** Concatenates tensors along a given dimension.

```
t1 = torch.randn(2, 3)
t2 = torch.randn(2, 3)
t_concat = torch.cat((t1, t2), dim=0)
```

h. **split():** Splits a tensor into chunks.

```
t = torch.randn(6, 3)
t_split = torch.split(t, 2)
```

i. **stack():** Stacks tensors along a new dimension.

```
t1 = torch.randn(3, 4)
t2 = torch.randn(3, 4)
t_stacked = torch.stack((t1, t2), dim=0)
```

3. Autograd

a. **Automatic Differentiation -** Autograd enables automatic computation of gradients, crucial for training neural networks.

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = x + 2
y.backward()
print(x.grad)
```

b. **`detach()`**: Detaches a tensor from the computation graph.

```
t = torch.randn(3, requires_grad=True)
t_detached = t.detach()
```

c. **`no_grad()`**: Disables gradient calculations within a block of code.

```
with torch.no_grad():
    t = torch.randn(3, requires_grad=True)
    t_no_grad = t * 2
```

d. **`retain_graph`**: Keeps the computation graph after the backward pass.

```
t = torch.tensor(2.0, requires_grad=True)
y = t * t
y.backward(retain_graph=True)
y.backward()
```

e. **Hessian Matrix Calculation**: Second-order partial derivatives.

```
def hessian(y, x):
    grad1 = torch.autograd.grad(y, x,
create_graph=True)[0]
    grad2 = [torch.autograd.grad(g, x,
retain_graph=True)[0] for g in grad1]
    return torch.stack(grad2)

t = torch.tensor([2.0, 3.0], requires_grad=True)
y = t[0] * t[1]
hess = hessian(y, t)
```

f. **`torch.autograd.grad()`**: Computes gradients of specified tensors.

```
t = torch.tensor(2.0, requires_grad=True)
y = t * t
grad = torch.autograd.grad(y, t)
```

4. NN Module

Building a Neural Network - The `nn.Module` is the base class for all neural network modules in PyTorch.

```
import torch.nn as nn
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(10, 5)

    def forward(self, x):
        x = self.fc1(x)
        return x

model = Net()
```

5. Optimizers

Defining Optimizers - Optimizers adjust the weights of neural networks to minimize the loss function.

```
import torch.optim as optim
optimizer = optim.SGD(model.parameters(),
lr=0.01, momentum=0.9)
```

6. DataLoader

Loading Data - The DataLoader is an iterator that provides batches of data from a dataset.

```
from torch.utils.data import DataLoader,
TensorDataset
dataset = TensorDataset(x, y)
dataloader = DataLoader(dataset,
batch_size=2, shuffle=True)
```

7. Loss Functions

Defining Loss - Loss functions measure the difference between the model's output and the target values.

```
criterion = nn.MSELoss()
loss = criterion(output, target)
```

8. Training Loop

Basic Training Loop - The training loop iteratively processes batches of data to update the model's parameters.

```
for epoch in range(10):
    for inputs, labels in dataloader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

9. CUDA Operations

Using GPU - PyTorch supports CUDA operations, allowing tensors to be moved to and processed on GPUs.

```
device = torch.device("cuda" if
torch.cuda.is_available() else "cpu")
model.to(device)
x = x.to(device)
```

10. Saving and Loading Models

Checkpointing - Saving and loading model checkpoints is essential for model persistence and resuming training.

```
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))
```

11. Pretrained Models

Using Pretrained Models - Pretrained models can be used for transfer learning, saving time and resources.

```
from torchvision import models
resnet = models.resnet18(pretrained=True)
```

12. Transformations

Data Transformations - Transformations are used to preprocess and augment data before feeding it to a model.

```
from torchvision import transforms
transform = transforms.Compose([transforms.ToTensor()])
```

13. Visualizing with TensorBoard

TensorBoard Integration - TensorBoard provides visualization tools to monitor and debug machine learning experiments.

```
from torch.utils.tensorboard import
SummaryWriter
writer = SummaryWriter()
writer.add_scalar('Loss/train', loss, epoch)
writer.close()
```

14. Random Number Generation

Setting Seeds - Setting seeds ensures reproducibility by fixing the randomness in PyTorch operations.

```
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed(42)
```

15. Distributed Training

Using Multiple GPUs - Distributed training allows parallel processing using multiple GPUs to accelerate training.

```
model = nn.DataParallel(model)
```

16. ONNX Export

Exporting Models to ONNX - ONNX enables interoperability between different deep learning frameworks.

```
torch.onnx.export(model, input, 'model.onnx')
```

17. Mixed Precision Training

AMP for Mixed Precision - Mixed precision training reduces memory usage and improves performance by using both 16-bit and 32-bit floating-point types.

```
from torch.cuda.amp import GradScaler,
autocast
scaler = GradScaler()

with autocast():
    output = model(input)
    loss = criterion(output, target)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

18. Model Quantization

Quantizing Models - Quantization reduces the model size and improves inference speed by converting weights to lower precision.

```
model_int8 =
torch.quantization.quantize_dynamic(model,
{nn.Linear}, dtype=torch.qint8)
```

19. Sparse Tensors

Working with Sparse Tensors - Sparse tensors store only non-zero elements, saving memory and computational resources.

```
sparse_tensor = torch.sparse_coo_tensor(indices,
values, [3, 3])
```

20. Parallelism with TorchScript

Scripting Models - TorchScript allows models to be optimized and run in a production environment with reduced overhead.

```
scripted_model = torch.jit.script(model)
```

21. TorchHub

Loading Models from TorchHub - TorchHub provides a repository of pre-trained models and other assets that can be easily loaded and used.

```
hub_model = torch.hub.load('pytorch/vision',
'resnet18', pretrained=True)
```

22. Execution Paradigms

a. **Eager Execution:** Default mode for immediate operation evaluation.

```
torch.set_grad_enabled(True)
t = torch.randn(2, 3, requires_grad=True)
```

b. **JIT (Just-In-Time) Compilation:** Compiles PyTorch models into TorchScript for optimized performance.

```
def f(x):
    return x * 2

t = torch.tensor([1.0, 2.0])
traced_f = torch.jit.trace(f, t)
t2 = traced_f(t)
```